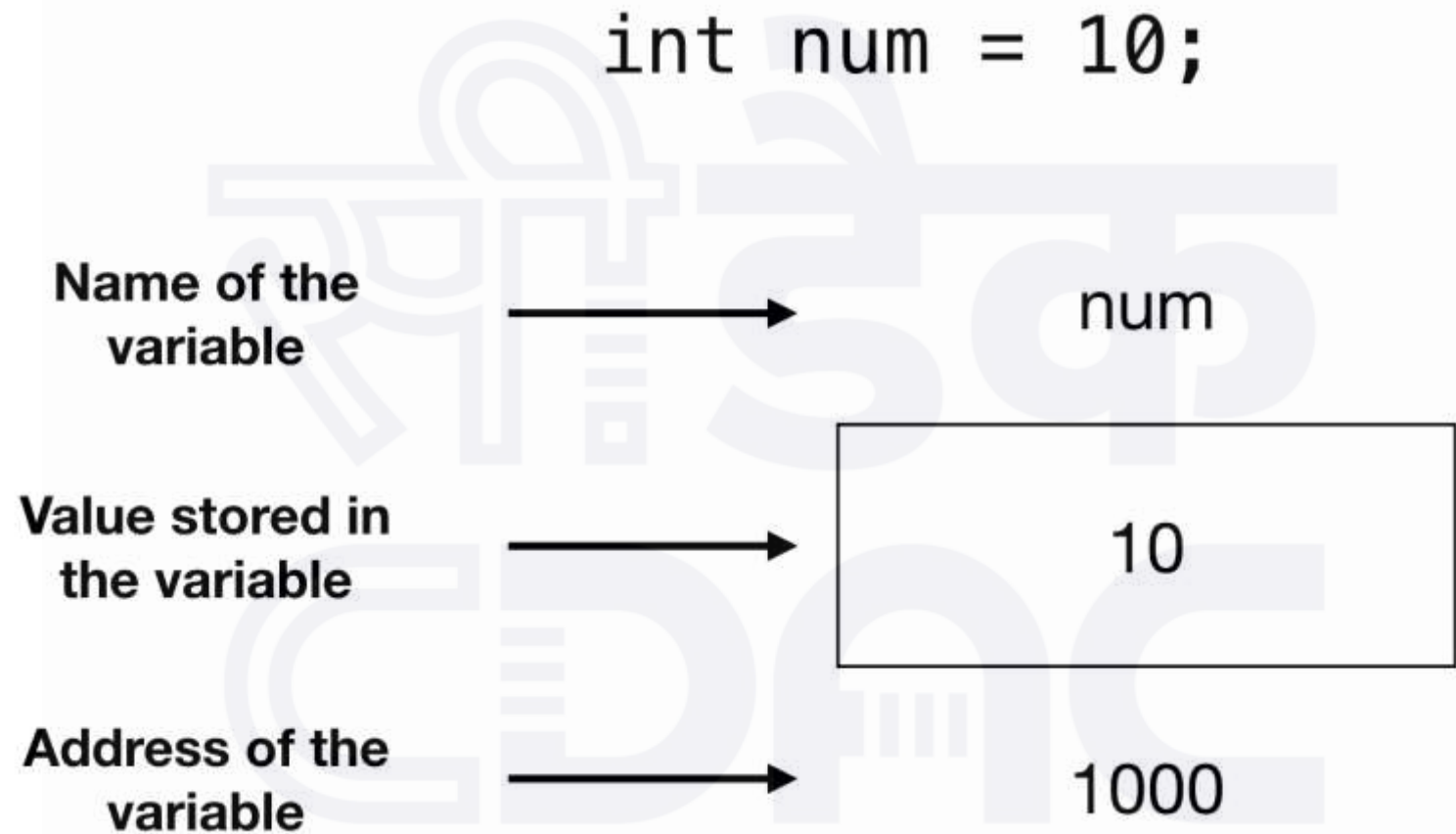


Pointers In C

Introduction to the pointers



What is a Pointer in C?

A pointer in C is a variable that stores the **memory address of another variable**.

A pointer variable points to a data type (like int) of the same type, and is created with the *** operator**.

They are a powerful feature used to manage memory, optimize performance, and enable dynamic data structures.

Basic Pointer Syntax:

Datatype should be same

Declare a pointer:

```
int *ptr; // Pointer to an integer
```

Assign the address of a variable to the pointer:

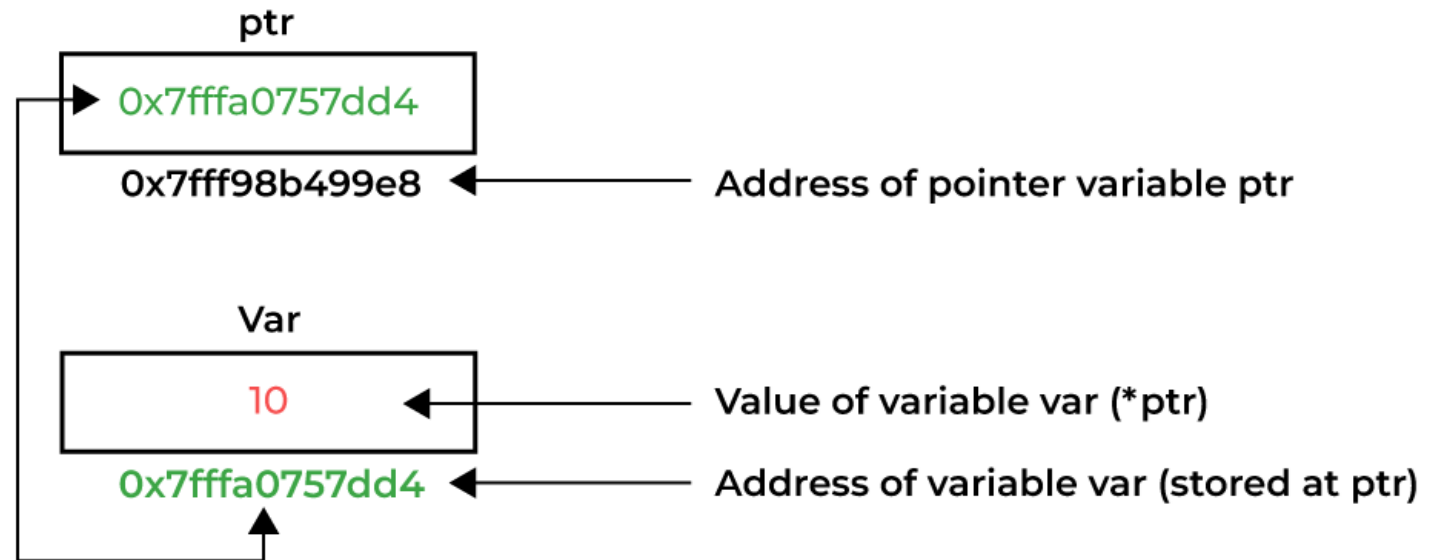
```
int x = 10; // Normal variable  
ptr = &x;   // & gives the address of 'x'
```

Access the value at the address stored by the pointer (dereference):

```
printf("%d", *ptr); // Prints the value of 'x' (10)
```

Basic Pointer Syntax:

Datatype should be same



Pointer Declaration

To declare a pointer, use the dereferencing operator (*) followed by the data type.

Syntax : type *var-name;

Examples:

```
int    *ip;    /* pointer to an integer */
```

```
double *dp;    /* pointer to a double */
```

```
float  *fp;    /* pointer to a float */
```

```
char   *ch     /* pointer to a character */
```

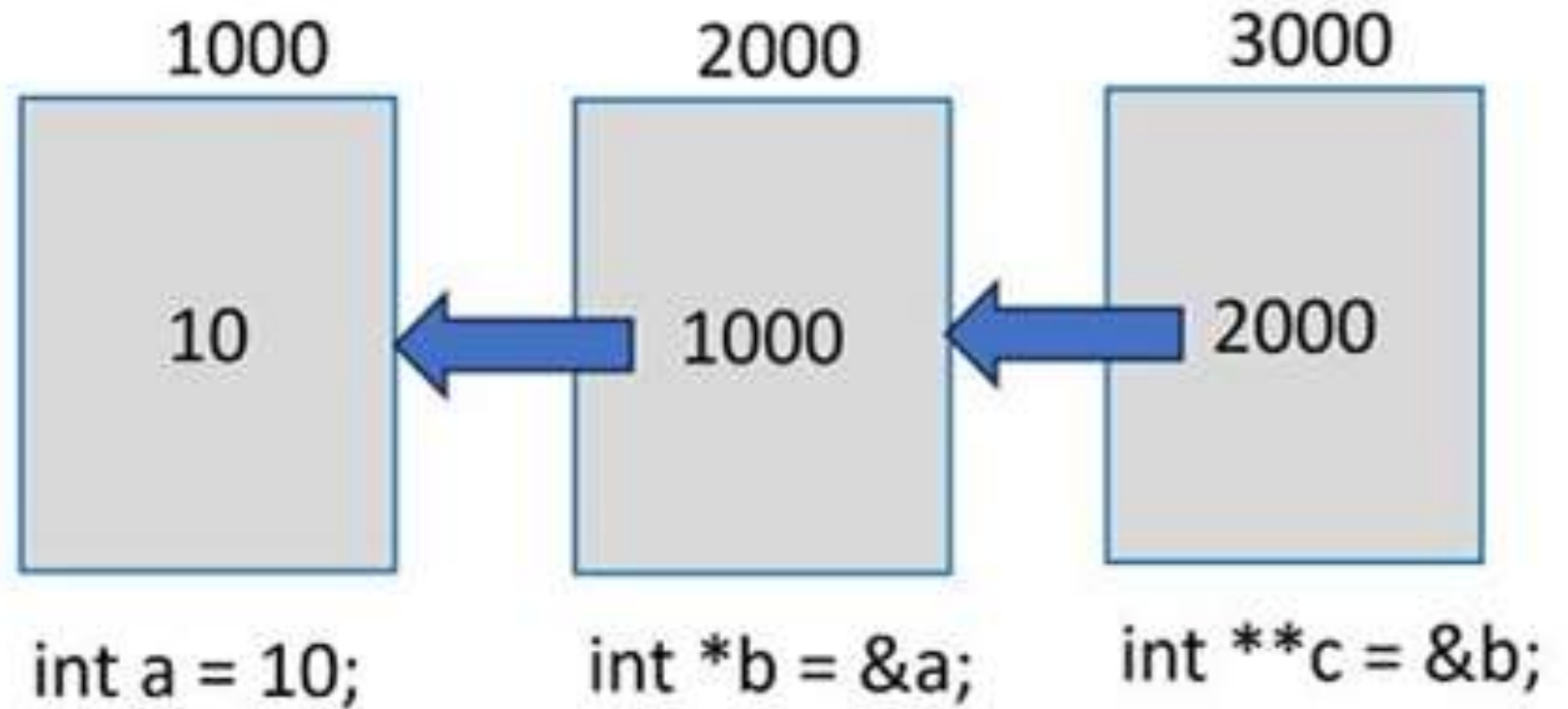
Pointer Initialization

- After declaring a pointer variable, you need to initialize it with the address of another variable using the address of (&) operator. This process is known as **referencing a pointer**.
- **Syntax:** `pointer_variable = &variable;`
- **Example :** `int x = 10;`
`int *ptr = &x;`

Pointer Assignment

```
int a = 10 ;  
int b = 11;  
int *p,*q;  
p = &a ; q= &b;  
*q = * p;
```

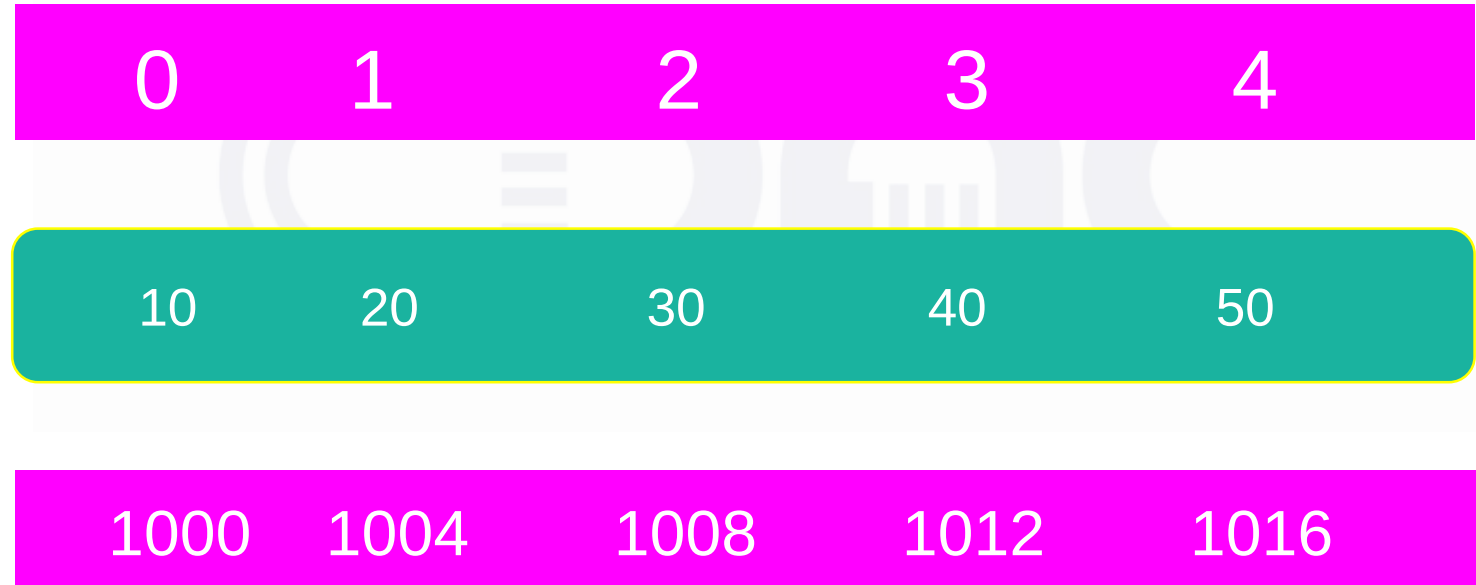
Double pointer



Pointer arithmetic

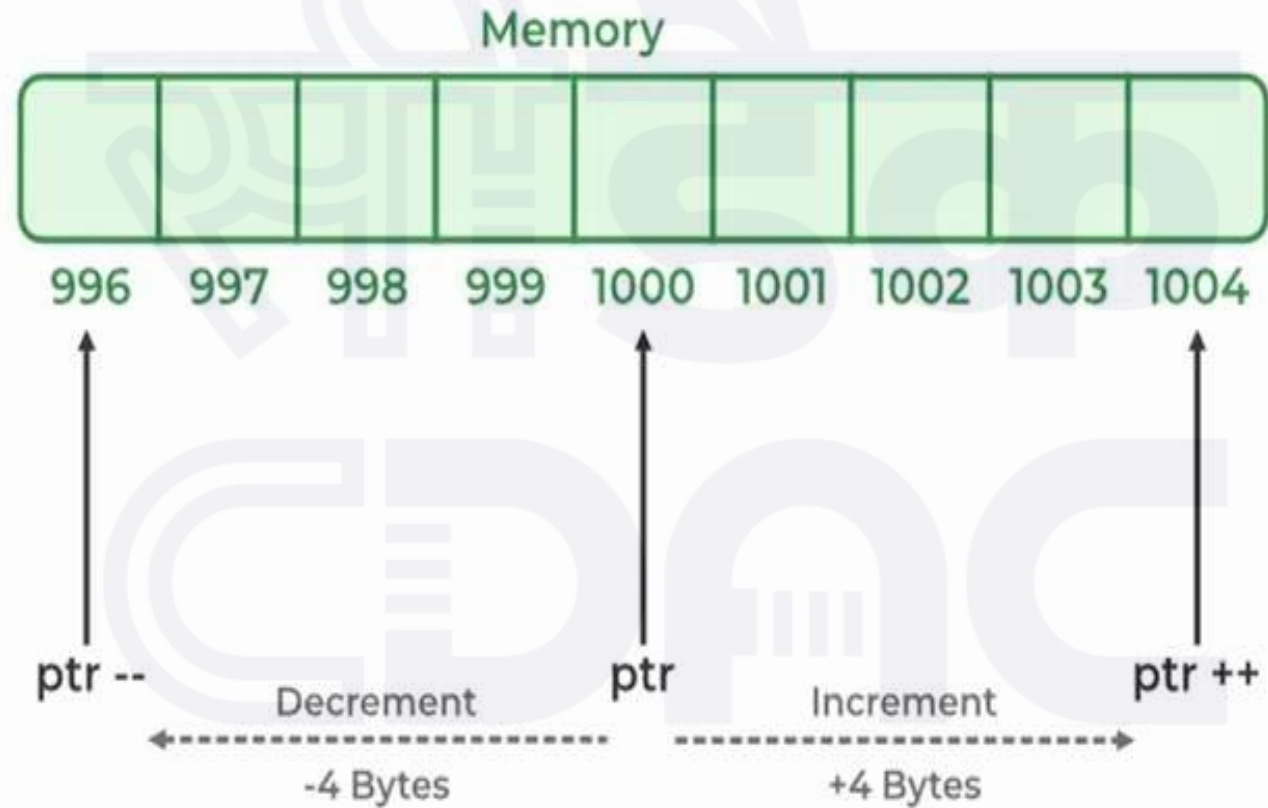
```
Int a[5] =  
{10,20,30,40,50};  
Int *p = &a[0];  
p=p+2
```

```
2  $\Rightarrow$  2X 4 bytes =8  
P  $\Rightarrow$  1000 +8 =1008  
P = &a[2];
```



Pointer arithmetic

Pointer Increment & Decrement



Pointer Comparison

```
// declaring array
```

```
int arr[5];
```

```
// declaring pointer to array  
name
```

```
int *ptr1 = arr;
```

```
// declaring pointer to  
first element
```

```
int *ptr2 = &arr[0];
```

```
if (ptr1 == ptr2) is  
always true
```

Pointer Comparison

```
#include <stdio.h>

int main(){
    // declaring array
    int arr[5];

    // declaring pointer to array name
    int *ptr1 = arr;
    // declaring pointer to first element
    int *ptr2 = &arr[0];

    if (ptr1 == ptr2) {
        printf("Pointer to Array Name and First Element "
               "are Equal.");
    }
    else {
        printf("Pointer to Array Name and First Element "
               "are not Equal.");
    }

    return 0;
}
```

Comparison to NULL

NULL — a special pointer value meaning "**points to nothing / not yet assigned a valid address.**" Comparing a pointer to NULL is the standard way to check, before dereferencing it, whether it's actually safe to use.

```
Int *ptr = NULL;
```

```
if (ptr == NULL)
```

Examples

```
#include <stdio.h>

int main() {
    int myAge = 43;    // An int variable
    int *ptr = &myAge; // A pointer variable, with the name ptr, that
                       // stores the address of myAge
    // Output the value of myAge (43)
    printf("%d\n", myAge);
    // Output the memory address of myAge (0x7ffe5367e044)
    printf("%p\n", &myAge);
    // Output the memory address of myAge with the pointer
    // (0x7ffe5367e044)
    printf("%p\n", ptr);
    return 0;}
```

%p format specifier, which is the correct, portable way to print a pointer's value (an address)

Pointer Operations

Pointer operations allow manipulation of memory addresses and values stored at those addresses.

Pointer Referencing

- Making a pointer variable to point other variables by providing address of that variable to the pointer is known as **referencing** of pointer.
- It is also known as **initialization** of pointers.
- Address-of-operator (&) Retrieves the memory address of a variable.

Examples

```
int a=5;
int *ptr;
ptr = &a;
//Here pointer ptr got address of variable a so, pointer ptr is now
pointing to variable a.
printf("Value of a: %d\n", *ptr); // Dereference to get the value of
'a'
*ptr = 30; // Modify the value of 'a' through the pointer
printf("Updated value of a: %d\n", a); // Output: 30
```

Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer.

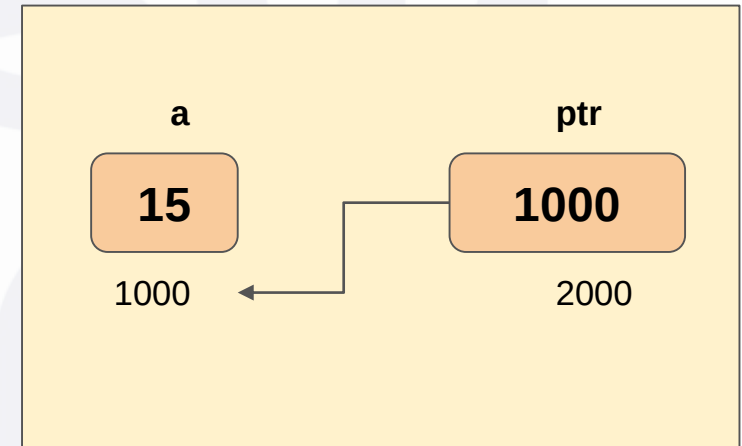
We use the same (*) dereferencing operator that we used in the pointer declaration.

Example:

```
int a = 15;
```

```
int *p = &a;
```

```
printf("Value of a: %d\n", *p); // 15
```



Key Points

- **Address operator(&)** used to **returns the address of a variable** or to access the address of a variable to a pointer.
- **Dereferencing operator(*)** used to declare pointer variable and **access the value stored in the address.**
- **%p** format specifier is used to **print the address stored in pointer variables.**
- Pointers are essential for **dynamic memory allocation**, providing control over memory usage with functions like **malloc**, **calloc**, and **free**.
- The memory address format will always be in **hexadecimal format**(starting with 0x).

Example

```
#include <stdio.h>

int main (){
    int *ptr ;
    int a=15;
    ptr = &a;
    printf("Value of a: %d\n", a);
    printf("Address of a: %p\n", &a);
    printf("Value of ptr: %p\n", ptr);
    printf("Address of ptr: %p\n", &ptr);
    *ptr = 20;
    printf("Value of a is now: %d\n",a );
    return 0;
}
```

Output:

Value of a: 15

address of a: 0x7ffdbf2f3df4

Value of ptr: 0x7ffdbf2f3df4

address of ptr:
0x7ffdbf2f3df8

Value of a is now :20

Problem:

Write a C program that uses a pointer to iterate through an array of integers and count the number of even and odd elements.

Solution

```
#include <stdio.h>
int main(){
    int n = 10;

    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8,
9, 10 };

    // Declaration of pointer
variable
    int *ptr;

    // Pointer points the first (0th
index)

    // element in an array
    ptr = arr;

    int count_even = 0;

    int count_odd = 0;

    for (int i = 0; i < n; i++) {
        if (*ptr % 2 == 0) {
            count_even++; }

        if (*ptr % 2 != 0) {
            count_odd++; }

        // Pointing to the next
        // element in an array
        ptr++; }

    printf("No of even elements in an array is :
%d", count_even);

    printf("\nNo of odd elements in an array is :
%d", count_odd);

}
```

Problem:

Use a pointer to find the sum and average of all elements in an integer array.

Problem:

Write a program using pointer arithmetic to count how many elements in an array are divisible by 5.

Problem:

Reverse an array in place using two pointers — one starting at the first element, one at the last, swapping and moving inward until they meet.